

LAPORAN WORKSHOP DEVOPS

BAB 2 : KONSEP CONTAINER DAN INSTALASI DOCKER



Oleh:

Zhulva Priya Abhipraya (3126640034)

Dosen Pengajar:

Dr. Ferry Astika Saputra, ST, M.Sc

**PROGRAM STUDI SARJANA TERAPAN LANJUT JENJANG TEKNIK
INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

2026

1. PENDAHULUAN DAN TUJUAN PRAKTIKUM

Dalam siklus hidup pengembangan perangkat lunak (SDLC) modern, efisiensi dan portabilitas menjadi parameter utama dalam pengiriman aplikasi. Teknologi kontainerisasi telah merevolusi cara aplikasi dikemas dan dijalankan dengan menawarkan isolasi proses yang jauh lebih ringan dibandingkan *Virtual Machine* (VM) tradisional. Berbeda dengan VM yang melakukan virtualisasi perangkat keras secara penuh dan menyertakan *guest kernel*, kontainer berbagi kernel host yang sama namun tetap terisolasi melalui mekanisme *namespaces* dan *control groups* (cgroups). Pendekatan ini menjamin konsistensi lingkungan eksekusi dari tahap pengembangan hingga produksi, sehingga memitigasi anomali "it works on my machine".

Praktikum ini dirancang untuk membekali mahasiswa dengan penguasaan teknis atas ekosistem Docker. Adapun tujuan praktikum ini mencakup:

- Memahami komponen inti arsitektur Docker (Client, Daemon, Registry, Image, dan Container).
- Melakukan instalasi dan konfigurasi Docker Engine pada lingkungan WSL2 dengan prinsip keamanan *rootless*.
- Menganalisis mekanisme kerja kontainer melalui eksekusi *image* publik dari Docker Hub.
- Membangun *bespoke image* yang efisien menggunakan instruksi terstruktur dalam Dockerfile.

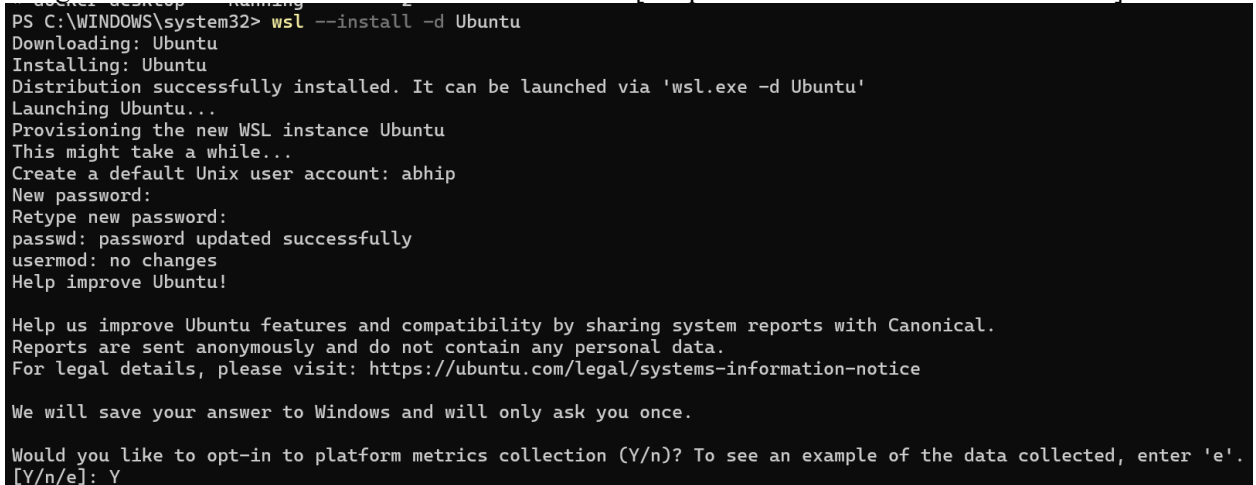
Pemahaman teoritis mengenai isolasi kernel ini menjadi fondasi penting sebelum memasuki tahap teknis penyiapan infrastruktur laboratorium pada sub-bab berikutnya.

2. MENYIAPKAN ARSITEKTUR LABORATORIUM DAN PRASYARAT LINGKUNGAN

Isolasi lingkungan kerja merupakan standar baku dalam administrasi sistem untuk menjaga integritas sistem operasi host. Dengan memanfaatkan *Windows Subsystem for Linux* (WSL2), praktikan dapat mengakses fitur-fitur kernel Linux yang diperlukan untuk kontainerisasi secara *native* di atas platform Windows. Selain aspek teknis, manajemen pengguna (*user management*) memegang peranan krusial; membatasi hak akses administratif melalui penggunaan *non-root user* adalah langkah preventif untuk meminimalkan *attack surface* pada infrastruktur DevOps.

Langkah Teknis Persiapan:

1. **Instalasi WSL:** Aktivasi lingkungan Linux dilakukan melalui terminal Windows dengan menginstal distribusi Ubuntu: `wsl --install -d Ubuntu` [Sisipkan Gambar: Proses Instalasi WSL]



```
PS C:\WINDOWS\system32> wsl --install -d Ubuntu
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: abhip
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!

Help us improve Ubuntu features and compatibility by sharing system reports with Canonical.
Reports are sent anonymously and do not contain any personal data.
For legal details, please visit: https://ubuntu.com/legal/systems-information-notice

We will save your answer to Windows and will only ask you once.

Would you like to opt-in to platform metrics collection (Y/n)? To see an example of the data collected, enter 'e'.
[Y/n/e]: Y
```

2. **Manajemen User:** Untuk memisahkan sesi kerja, dibuat pengguna baru bernama devopsabhi. Penggunaan akun non-administrator ini bertujuan untuk mencegah potensi kerusakan sistem jika terjadi kesalahan konfigurasi atau eksploitasi pada level aplikasi. `sudo adduser devopsabhi` [Sisipkan Gambar: Pembuatan User devopsabhi]

```

abhip@ASUS:/mnt/c/WINDOWS/system32$ sudo adduser devopsabhi
[sudo: authenticate] Password:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for devopsabhi
Enter the new value, or press ENTER for the default
    Full Name []: devopsabhi
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
abhip@ASUS:/mnt/c/WINDOWS/system32$ su - devopsabhi
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep  8 19:28:52 +07 2026

System load:  0.23           Processes:            33
Usage of /:   0.1% of 1006.85GB Users logged in:      0
Memory usage: 6%           IPv4 address for eth0: 172.20.20.172
Swap usage:   0%

```

3. **Akses Sesi Terisolasi:** Berdasarkan standar laboratorium, akses ke sesi pengguna dilakukan menggunakan perintah `machinectl shell devopsabhi@` untuk memastikan terciptanya sesi terminal baru yang bersih dan terpisah dari sesi pengguna utama.
4. **Inisialisasi Direktori Kerja:** Ruang lingkup proyek dibatasi pada direktori spesifik untuk menjaga keteraturan artefak: `mkdir -p ~/docker-lab/bab-2`

Kesiapan lingkungan host ini menandakan bahwa infrastruktur telah siap untuk proses instalasi dan konfigurasi *container engine*.

3. INSTALASI DAN VERIFIKASI DOCKER

Docker Engine bertindak sebagai layanan latar belakang (*daemon*) yang mengelola seluruh siklus hidup kontainer. Pada praktikum ini, implementasi Docker dilakukan menggunakan **Rootless Mode**. Berbeda dengan instalasi standar yang berjalan dengan hak akses root, *rootless mode* memungkinkan Docker daemon berjalan di bawah konteks pengguna biasa, yang secara signifikan mengurangi risiko eskalasi hak akses jika kontainer berhasil dikompromi.

Langkah Verifikasi Sistem:

1. **Akses User Praktikum:** Seluruh operasional dijalankan di bawah identitas devopsabhi pada host ASUS untuk memastikan prinsip *least privilege*.

```

abhip@ASUS:/mnt/c/WINDOWS/system32$ sudo adduser devopsabhi
[sudo: authenticate] Password:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for devopsabhi
Enter the new value, or press ENTER for the default
    Full Name []: devopsabhi
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
abhip@ASUS:/mnt/c/WINDOWS/system32$ su - devopsabhi
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Sep  8 19:28:52 +07 2026

System load:  0.23           Processes:            33
Usage of /:   0.1% of 1006.85GB Users logged in:       0
Memory usage: 6%           IPv4 address for eth0: 172.20.20.172
Swap usage:   0%

```

2. **Verifikasi Container Pertama:** Eksekusi perintah docker run hello-world dilakukan untuk memvalidasi fungsionalitas sistem. Mekanisme internal yang terjadi meliputi:
 - **Handshake Client-Daemon:** Docker Client mengirimkan instruksi ke Daemon.
 - **Image Pulling:** Daemon memeriksa ketersediaan *image* di lokal; jika tidak ditemukan, Daemon melakukan *pulling* dari Docker Hub.
 - **Container Initialization:** Daemon membuat kontainer baru dan mengalokasikan sumber daya yang diperlukan.
 - **Output Streaming:** Kontainer mengeksekusi proses internal dan mengirimkan hasilnya kembali ke Client melalui *standard streams* (STDOUT).
3. **Analisis Output:**

Error code: WSL/WSL_E_USER_NOT_FOUND

PS C:\WINDOWS\system32> wsl -d Ubuntu -u devopsabhi

devopsabhi@ASUS:/mnt/c/WINDOWS/system32\$ cd ~/docker-lab/bab-2

devopsabhi@ASUS:~/docker-lab/bab-2\$ docker run hello-world

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

devopsabhi@ASUS:~/docker-lab/bab-2\$

devopsabhi@ASUS:~/docker-lab/bab-2\$ docker run --name web-public -d -p 8082:80 nginx:1.26

3fb585e807c0f250d84639488725851bece619ed68a5b12a4732d51a2be1fe42

devopsabhi@ASUS:~/docker-lab/bab-2\$ docker ps

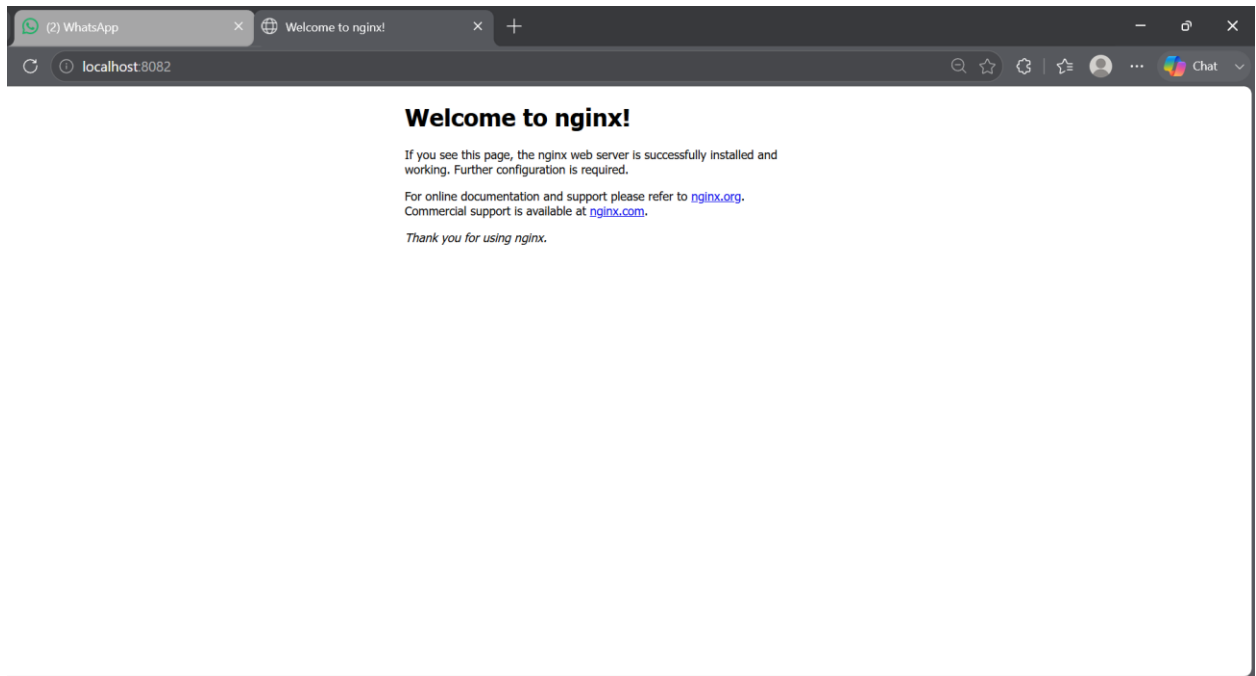
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3fb585e807c0	nginx:1.26	"/docker-entrypoint..."	16 seconds ago	Up 15 seconds	0.0.0.0:8082->80/tcp	web-public

devopsabhi@ASUS:~/docker-lab/bab-2\$ curl localhost:8082

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```



4. CONTAINER NGINX DAN CUSTOM DOCKERFILE

Dalam arsitektur *microservices*, pemilihan antara *off-the-shelf image* (publik) dan *bespoke image* (kustom) merupakan keputusan strategis. Image publik memberikan kecepatan *deployment*, namun image kustom memungkinkan optimasi melalui *image layering*—di mana setiap instruksi dalam Dockerfile menciptakan lapisan baru yang *read-only*, membantu efisiensi *caching* dan pengurangan ukuran artefak aplikasi.

Langkah-Langkah Implementasi:

1. **Deployment Nginx Public:** Menjalankan layanan Nginx versi 1.26 standar dengan nama web-public pada port **8082**: `docker run -d --name web-public -p 8082:80 nginx:1.26` Pemetaan port `-p 8082:80` mengarahkan trafik dari port host 8082 ke port internal kontainer 80, sementara mode `-d` (*detached*) memastikan layanan berjalan di latar belakang. [Sisipkan Gambar: Running Nginx Public]

```
devopsabhi@ASUS:~/docker-lab/bab-2$ docker pull nginx:1.26
1.26: Pulling from library/nginx
4fd410795c0f: Pull complete
8a628cdd7ccc: Pull complete
6923759e66ab: Pull complete
d44088bb6ae8: Pull complete
9ebfb40fb06b: Pull complete
7a0654aeb922: Pull complete
5e98d206134b: Pull complete
Digest: sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
Status: Downloaded newer image for nginx:1.26
docker.io/library/nginx:1.26
```

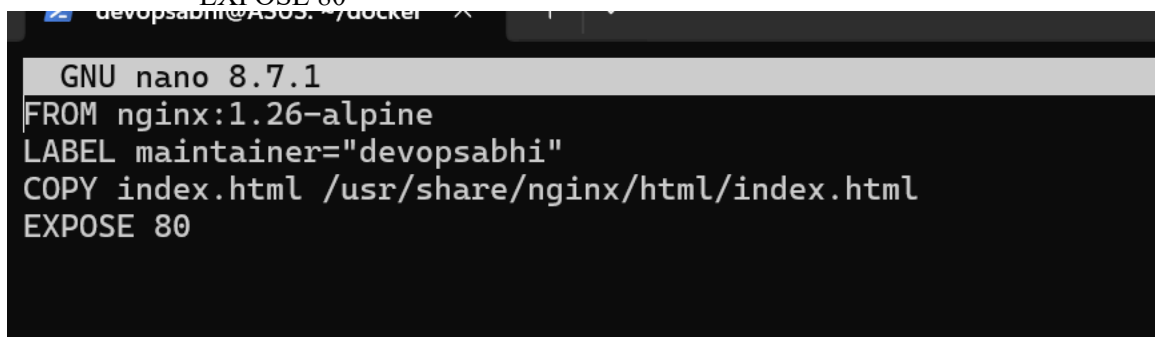
```
devopsabhi@ASUS:~/docker-lab/bab-2$ docker run --name web-public -d -p 8082:80 nginx:1.26
3fb585e807c0f250d84639488725851bece619ed68a5b12a4732d51a2be1fe42
devopsabhi@ASUS:~/docker-lab/bab-2$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                NAMES
3fb585e807c0   nginx:1.26 "/docker-entrypoint..." 16 seconds ago Up 15 seconds    0.0.0.0:8082->80/tcp   web-public
devopsabhi@ASUS:~/docker-lab/bab-2$ curl localhost:8082
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

2. **Pengembangan Web Kustom:** Untuk keperluan personalisasi, dibuat file index.html dengan konten: *"Halo, ini adalah web kustom pertama saya di Docker! - devopsabhi"*. Struktur Dockerfile disusun sebagai berikut:

```
FROM nginx:1.26-alpine
LABEL maintainer="Zhulva Priya Abhipraya <3126640034@student.pens.ac.id>"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```



```
devopsabhi@ASUS:~/docker-lab/bab-2$ nano Dockerfile
GNU nano 8.7.1
FROM nginx:1.26-alpine
LABEL maintainer="devopsabhi"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

- **Analisis Teknis:** Penggunaan base image alpine sangat direkomendasikan karena ukurannya yang minimal dan jumlah kerentanan (vulnerabilities) yang lebih sedikit. Instruksi COPY menciptakan lapisan baru yang menimpa konten default Nginx dengan konten kustom.
1. **Build & Run:** Membangun image kustom dengan perintah: `docker build -t web-custom:v1 .` Selanjutnya, kontainer dijalankan dengan nama web-personal pada port **8081**: `docker run -d --name web-personal -p 8081:80 web-custom:v1` [Sisipkan Gambar: Build dan Running Web Kustom]

```

devopsabhi@ASUS:~/docker-lab/bab-2$ echo "<h1>Halo, ini adalah web kustom pertama saya di Docker! - devopsabhi</h1>" > index.html
devopsabhi@ASUS:~/docker-lab/bab-2$ nano Dockerfile
devopsabhi@ASUS:~/docker-lab/bab-2$ docker build -t web-custom:v1 .
[+] Building 28.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 149B
=> [internal] load metadata for docker.io/library/nginx:1.26-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 111B
=> [1/2] FROM docker.io/library/nginx:1.26-alpine@sha256:1eadbb07820339e8bbfed18c771691970baee292ec4ab2558f1453d26153e22d
=> resolve docker.io/library/nginx:1.26-alpine@sha256:1eadbb07820339e8bbfed18c771691970baee292ec4ab2558f1453d26153e22d
=> sha256:5ab072e0bc4f67351f9c935c8834e3elbcacbe013a124ef309607d6210f218d 15.19MB / 15.19MB
=> sha256:a7c661bcc4ef10a5e7ee3b4f80a8d77754a4e6b1a598271b70bd220d804316eb 1.40kB / 1.40kB
=> sha256:c35c2aa19c882e9ca054353d84a4b58904f72807055c441be84f27401714f25d 1.21kB / 1.21kB
=> sha256:d987ab110a5ba4aebf9df7115759f01e8a5d8be0a70045f4dc7a5b959be1b04c 404B / 404B
=> sha256:b865133baae33f196ce13e6d5cfc16e276b640bbca95329625fbb6a3f13855 956B / 956B
=> sha256:5e18ef93c435d9cd1c48d1ae900cf7e92ee61ae11be6b3bdefc899f4be8efad9 628B / 628B
=> sha256:914d0eaf60479e4b17101b02ed1e82400c02231d11c32276fb24867f5f39e97b 1.76MB / 1.76MB
=> sha256:0a9a5dfd008f05ebc27e4790db0709a29e527690c21bcbcd01481eae6bb49dc 3.63MB / 3.63MB
=> extracting sha256:0a9a5dfd008f05ebc27e4790db0709a29e527690c21bcbcd01481eae6bb49dc
=> extracting sha256:914d0eaf60479e4b17101b02ed1e82400c02231d11c32276fb24867f5f39e97b
=> extracting sha256:5e18ef93c435d9cd1c48d1ae900cf7e92ee61ae11be6b3bdefc899f4be8efad9
=> extracting sha256:b865133baae33f196ce13e6d5cfc16e276b640bbca95329625fbb6a3f13855
=> extracting sha256:d987ab110a5ba4aebf9df7115759f01e8a5d8be0a70045f4dc7a5b959be1b04c
=> extracting sha256:c35c2aa19c882e9ca054353d84a4b58904f72807055c441be84f27401714f25d
=> extracting sha256:a7c661bcc4ef10a5e7ee3b4f80a8d77754a4e6b1a598271b70bd220d804316eb
=> extracting sha256:5ab072e0bc4f67351f9c935c8834e3elbcacbe013a124ef309607d6210f218d
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image

```

Meskipun alur kerja di atas terlihat linear, praktisi DevOps seringkali menghadapi kendala sintaksis dan logika yang memerlukan kemampuan diagnosis yang tajam.

```

devopsabhi@ASUS:~/docker-lab/bab-2$ docker run --name web-personal -d -p 8081:80 web-custom:v1
6202572f917e6c86ff9bbe76ecbd40fad30e0dfa422f8ffe938f2ce83fc0134f
devopsabhi@ASUS:~/docker-lab/bab-2$ url localhost:8081
Command 'url' not found, did you mean:
  command 'ul' from deb bsdxtrautils (2.41.3-3ubuntu2)
  command 'curl' from deb curl (8.18.0-1ubuntu2)
  command 'erl' from deb erlang-base (1:27.3.4.6+dfsg-1)
Try: apt install <deb name>
devopsabhi@ASUS:~/docker-lab/bab-2$ curl localhost:8081
<h1>Halo, ini adalah web kustom pertama saya di Docker! - devopsabhi</h1>

```

Halo, ini adalah web kustom pertama saya di Docker! - devopsabhi

5. ANALISIS & PROBLEM SOLVING

Kemampuan *troubleshooting* merupakan pembeda utama antara operator dan engineer. Dalam praktikum ini, ditemukan beberapa kegagalan instruksi yang memberikan pelajaran berharga mengenai ketelitian sintaksis dan pemahaman alur kerja CLI Docker.

Analisis Kasus Kegagalan:

- Kesalahan Sintaks pada Dockerfile:**
 - Masalah:** Saat proses *build*, sistem melaporkan kegagalan pada instruksi FROM. Teridentifikasi adanya typo: FROM nginx.1.26-alpine (menggunakan titik sebagai pemisah versi, bukan titik dua).
 - Diagnosa:** Docker Engine menginterpretasikan nginx.1.26-alpine sebagai nama repositori yang tidak valid atau tidak ditemukan karena hilangnya penanda tag (:).
 - Resolusi:** Melakukan koreksi pada Dockerfile menjadi FROM nginx:1.26-alpine.
- Kesalahan Argumen Build (Missing Build Context):**
 - Masalah:** Perintah docker build -t web-custom:v1 menghasilkan pesan error yang menyatakan kekurangan argumen.

- **Diagnosa:** Setiap perintah *build* membutuhkan "build context", yaitu lokasi direktori tempat Docker mencari file yang diperlukan. Praktikan lupa menyertakan tanda titik (.) di akhir perintah.
 - **Resolusi:** Menambahkan argumen . untuk merepresentasikan direktori saat ini sebagai konteks *build*.
3. **Eksplorasi Orchestration:** Ditemukan pula kendala saat mencoba menjalankan docker compose tanpa adanya file docker-compose.yml. Hal ini menegaskan bahwa otomatisasi memerlukan file konfigurasi deklaratif yang valid agar mesin orkestrasi dapat berfungsi.

Pengalaman ini menunjukkan bahwa pesan *error* pada CLI adalah panduan diagnostik yang akurat jika dipahami secara teknis.

6. RISIKO KEAMANAN & REKOMENDASI

Mengacu pada prinsip DevSecOps, keamanan harus diintegrasikan sedini mungkin dalam infrastruktur kontainer. Infrastruktur yang telah dibangun dianalisis memiliki postur keamanan yang lebih baik karena penggunaan mode *rootless*, namun tetap memiliki risiko operasional.

Analisis Risiko: Risiko paling kritis dalam ekosistem kontainer adalah "Container Escape". Jika kontainer berjalan dengan hak akses **root** dan terjadi eksploitasi pada aplikasi, penyerang dapat menembus isolasi kernel dan mengambil alih kendali penuh atas sistem host. Selain itu, penggunaan *image* publik yang tidak terverifikasi dapat membawa *malware* atau dependensi usang ke dalam jaringan internal.

Rekomendasi Strategis (Tabel Mitigasi):

Risiko Teridentifikasi	Tindakan Mitigasi (Rekomendasi)
Container Escape via Root Daemon	Mempertahankan penggunaan Docker Rootless Mode secara konsisten untuk membatasi hak akses daemon.
Vulnerable Image Layers	Selalu menggunakan base image minimal seperti Alpine Linux untuk memperkecil <i>attack surface</i> .
Supply Chain Attack	Menggunakan alat pemindai kerentanan seperti Trivy pada setiap image sebelum dilakukan <i>deployment</i> .
Privileged Container	Menghindari penggunaan flag <code>--privileged</code> yang dapat memberikan akses perangkat keras host secara tidak terkendali.

Praktikum ini berhasil membuktikan bahwa teknologi kontainer, jika dikonfigurasi dengan prinsip keamanan yang tepat, mampu memberikan efisiensi tinggi tanpa mengorbankan integritas sistem. Penguasaan atas Dockerfile dan kemampuan *troubleshooting* yang telah dipraktikkan menjadi modal utama dalam membangun alur kerja DevOps yang andal.